esprit

Ecole Supérieure Privée
d'Ingénierie et de Technologies

PLATEFORME D'ENTRAÎNEMENT ET D'ÉVALUATION DE L'EXPRESSION ORALE EN FRANÇAIS ET EN ANGLAIS



Notre equipe

Adem Jazi

Mariem Madeni

Oussema Ajmi

Med Selim Hmili

Chedli gharieni

Arij Sellami



Plan

1. Introduction
2. user-progress service
3. exam-content service
4. vocabulary-service
5. simulation-service
6. feedback-service
7. certification-service
8. analysis-service
9. Les diagrammes
10. les interfaces



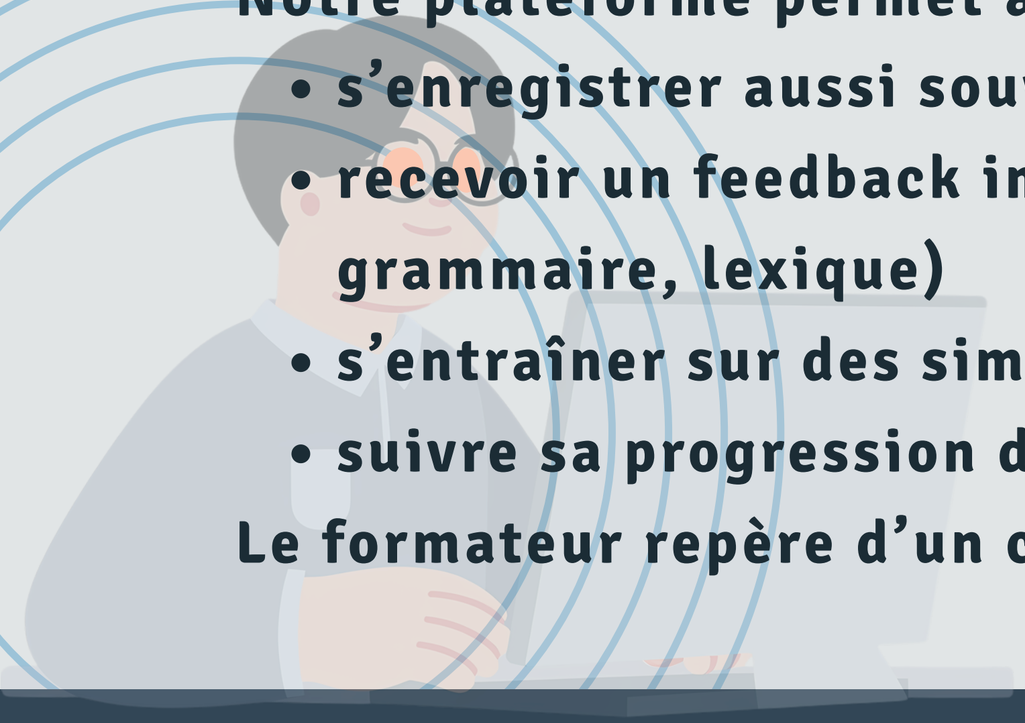
Introduction

L’oral est la compétence la plus déterminante aux examens de langue, et pourtant la moins entraînée.

Notre plateforme permet à l’apprenant de :

- s’enregistrer aussi souvent qu’il le souhaite
- recevoir un feedback immédiat par IA (prononciation, fluidité, grammaire, lexique)
- s’entraîner sur des simulations fidèles aux examens officiels
- suivre sa progression du niveau B1 au niveau C2 du CECRL

Le formateur repère d’un coup d’œil les difficultés de son groupe.



1-USER PROGRESS SERVICE

Gestion des utilisateurs et suivi de la progression des apprenants

RÔLE ET FONCTIONNALITÉS

Ce microservice centralise les informations des apprenants et permet de suivre leur évolution au sein de la plateforme.

- Gestion des utilisateurs et des rôles
- Organisation des groupes et des établissements
- Suivi du niveau linguistique selon le CECRL
- Consultation et mise à jour de la progression
- Gestion des alertes des apprenants

ENTITÉS PRINCIPALES

User

Group

Establishment

Progress

Alert

2 - Microservice Exam Content

exam-content-service

Rôle

Gérer les sujets d’oral par niveau et par langue, et fournir les grilles d’évaluation et descripteurs CECRL.

Technologies

Spring Boot | **H2** | **Swagger** | **Eureka**

Communication

- Appelé par simulation-service (Feign)
- N’appelle aucun autre service
- Ne produit aucun événement
- Ne consomme aucun événement

Entités principales

Subject
Sujet d’oral

Grid
Grille par niveau

Descriptor
Descripteurs CECRL

Criterion
Critères d’évaluation

Endpoints

`GET /api/subjects?level=B2&lang=EN`
Sujets par niveau et langue

`GET /api/subjects/random`
Tirage au sort d’un sujet

`POST /api/subjects`
Ajouter un sujet (formateur)

`GET /api/grids/{level}`
Grille d’évaluation d’un niveau

3 - Microservice Recording

recording-service

Rôle

Recevoir les enregistrements audio, conserver leurs métadonnées et obtenir la transcription via un moteur Speech-to-Text externe.

Technologies

Spring Boot **H2** **Swagger** **Eureka**

Communication

- Aucun appel Feign vers un autre service
- Appelle Whisper ou Vosk en HTTP
- Produit RecordingTranscribed
- Consommé par analysis-service

Entités principales

Recording
Fichier audio et métadonnées

Transcript
Texte transcrit de l'enregistrement

Endpoints

`POST /api/recordings`
Envoyer un enregistrement audio

`GET /api/recordings/{id}`
Métadonnées d'un enregistrement

`GET /api/recordings/{id}/transcript`
Transcription d'un enregistrement

4 - Microservice Certification

certification-service

Rôle

Gérer le catalogue des certifications, mesurer l'écart avec le niveau visé et délivrer les attestations de niveau.

Technologies

Spring Boot **MySQL** **Swagger** **Eureka**

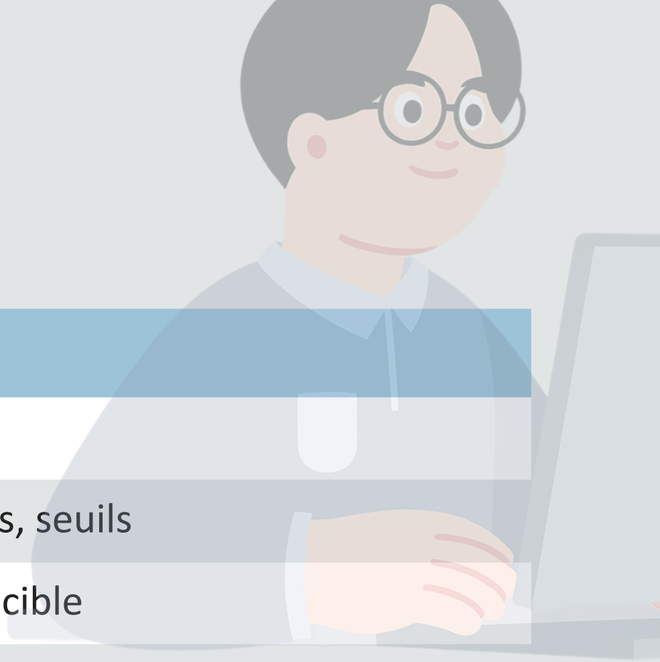
Entités principales

Certification
DELF, DALF, TCF, Cambridge, IELTS, TOEFL

ExamFormat
Format, durée, niveau requis, seuils

Attestation
Attestation de niveau délivrée

API et communication



Endpoint	Description
`GET /api/certifications`	Catalogue des certifications
`GET /api/certifications/{id}`	Format, durée, niveau requis, seuils
`GET /api/certifications/{id}/readiness/{userId}`	Écart entre niveau actuel et cible
`POST /api/attestations`	Délivrer une attestation
`GET /api/attestations/user/{userId}`	Attestations d'un apprenant

Synchrone (OpenFeign)

- Appelle user-progress-service pour lire la progression
- Appelé par simulation-service (déroulé, durée)

Asynchrone (événements)

- Consomme SimulationFinished
- Produit AttestationIssued

5 - Microservice Simulation

simulation-service

Rôle

Conduire une épreuve blanche : tirage du sujet, chronomètre, relances d'un examinateur virtuel, note estimée par critère CECRL.

Technologies

Spring Boot · MySQL · Eureka · OpenFeign · RabbitMQ · Swagger · Gemini

MySQL : données relationnelles (sessions, tours, scores).

Communication

- Appelle exam-content-service (sujet et grille)
- Appelle recording-service (transcription)
- Appelle user-progress-service (profil)
- Produit l'événement SimulationFinished

Entités principales

SimulationSession
Épreuve blanche

SimulationTurn
Tour de parole

SimulationScore
Note par critère

SimulationStatus
Cycle de vie

Endpoints

`POST /api/simulations`
Démarrer une épreuve

`GET /api/simulations/{id}`
État de l'épreuve

`POST /api/simulations/{id}/turns`
Envoyer une réponse

`POST /api/simulations/{id}/finish`
Terminer et noter

API et communication

Endpoint	Description	Codes
POST /api/simulations	Démarrer une épreuve (tirage du sujet)	201, 400, 404
GET /api/simulations/{id}	État de l'épreuve	200, 404
POST /api/simulations/{id}/turns	Envoyer une réponse (audioId), recevoir la relance	201, 404, 409
POST /api/simulations/{id}/finish	Terminer et noter	200, 404, 409
GET /api/simulations/{id}/result	Note estimée par critère	200, 404, 409
GET /api/simulations/me	Historique de l'apprenant	200

Synchrone (OpenFeign)

- exam-content : sujet, grille. Si absent : banque locale de sujets
- recording : transcription. Si absent : 503, l'apprenant renvoie
- certification : durées. Si absent : durées par défaut
- Réponse nécessaire immédiatement

Asynchrone (RabbitMQ)

- Produit SimulationFinished : l'apprenant n'attend pas l'analyse
- Si RabbitMQ est arrêté : note sauvegardée, événement republié

6 - Microservice Feedback

feedback-service

Rôle

Générer des retours pédagogiques détaillés (grammaire, lexique), proposer des reformulations et des exercices ciblés, et intégrer la révision du formateur.

Technologies

Spring Boot | MySQL | Swagger | Eureka

Communication

- Appelé par user-progress-service (Feign)
- N'appelle aucun autre service
- Consomme AnalysisCompletedEvent
- Produit FeedbackValidatedEvent

Entités principales

Feedback
Synthèse, scores et niveau

Correction
Fautes et reformulations

Exercise
Remédiation personnalisée

TrainerReview
Validation formateur

Endpoints

`GET /api/feedbacks/hello`
Vérification de l'état du microservice

`GET /api/feedbacks/{id}`
Consulter le feedback et ses corrections

`POST /api/feedbacks/generate`
Génération automatique du feedback post-analyse

`POST /api/feedbacks/{id}/reviews`
Validation et notes du formateur

7 - Microservice Analysis

analysis-service

Rôle

Analyser la réponse orale d'un apprenant : mots, phonèmes, pauses, courbe d'intonation et score par critère CECRL.

Technologies

Spring Boot · Swagger · Eureka · OpenFeign · MongoDB · RabbitMQ

MongoDB : documents imbriqués (mots, phonèmes, pauses).

Communication

- Consomme SimulationFinished
- Appelle recording-service (transcription, audio)
- Appelle exam-content-service (grille, critères)
- Produit l'événement AnalysisCompleted



Entités principales

Analysis
Analyse d'une réponse

WordResult
Mots et phonèmes

Pause / IntonationCurve
Rythme, intonation

CriterionScore
Note par critère

Endpoints

`GET /api/analyses/{id}`
Analyse complète

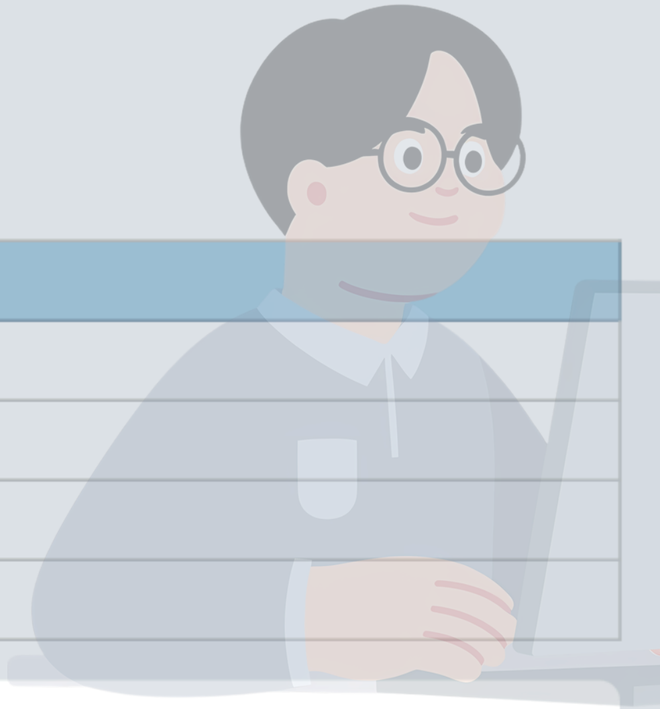
`GET /api/analyses?simulationId=`
Analyse d'une épreuve

`GET /api/analyses/{id}/criteria`
Notes par critère

`GET /api/analyses/user/{userId}`
Historique d'un apprenant

API et communication

Endpoint	Description	Codes
GET /api/analyses/{id}	Analyse complète	200, 404
GET /api/analyses?simulationId=	Analyse d'une épreuve	200, 404
GET /api/analyses/{id}/criteria	Notes par critère	200, 404
GET /api/analyses/user/{userId}	Historique d'un apprenant	200



Synchrone (OpenFeign)

- recording : transcription, audio
- exam-content : grille, critères
- Si l'un est absent : message rejeté, retenté, puis file des échecs (DLQ)
- Le calcul a besoin des données immédiatement

Asynchrone (RabbitMQ)

- Consomme SimulationFinished : l'analyse est longue, créée par l'événement (pas de POST)
- Produit AnalysisCompleted
- Si le service est arrêté : les messages attendent dans sa file
- Idempotence : un eventId déjà traité est ignoré

Diagramme de classe global



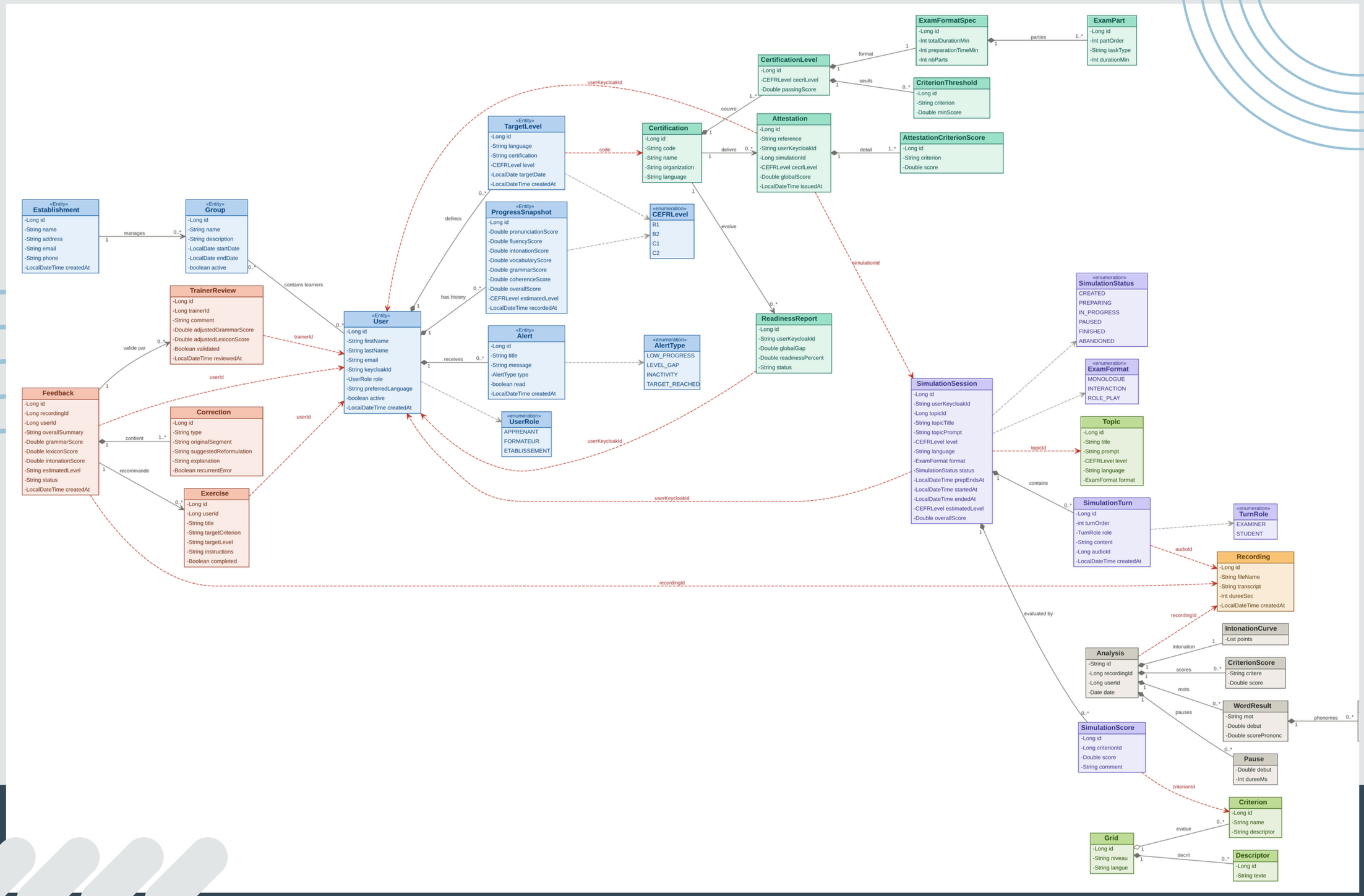
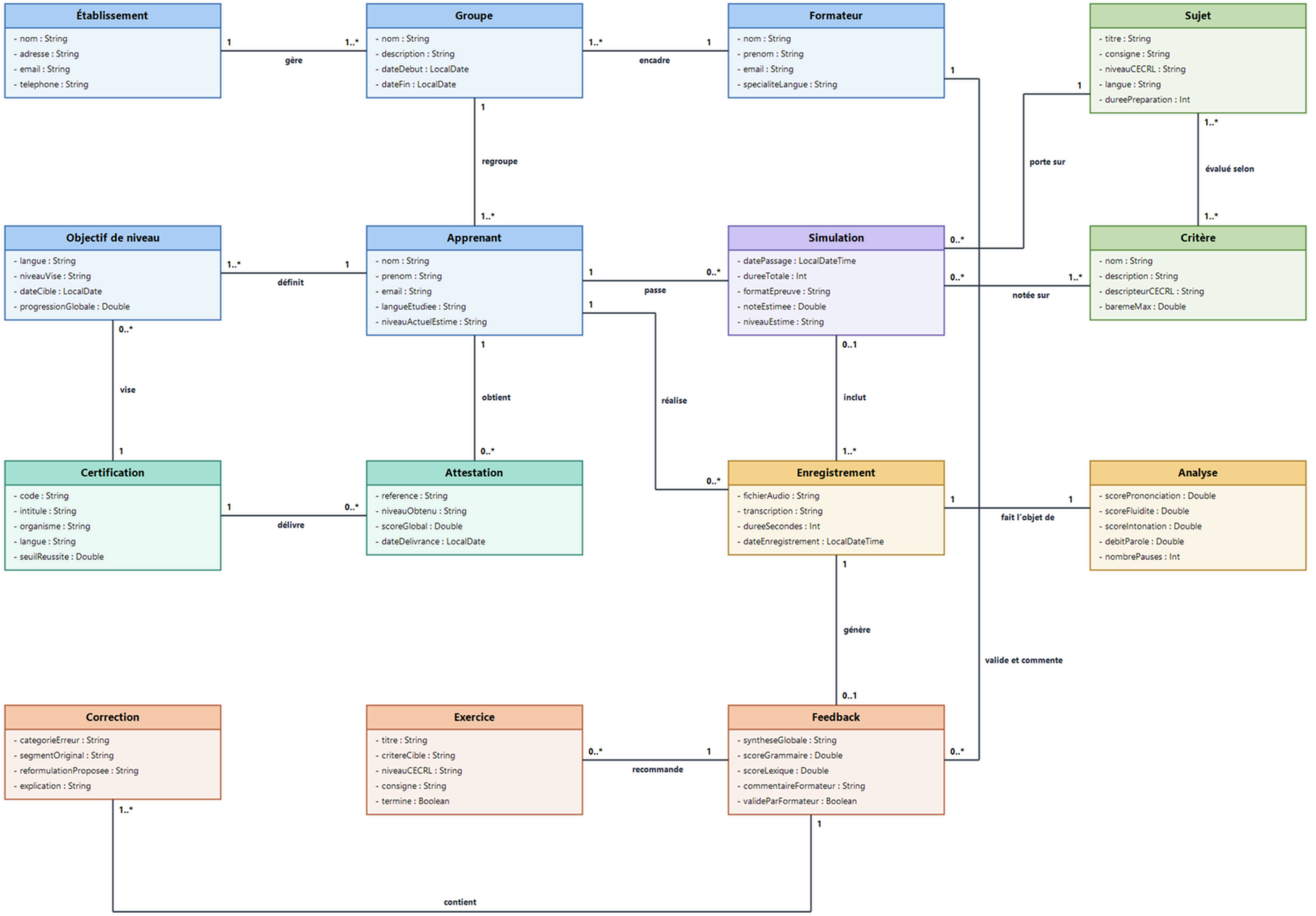

«Entity»
Establishment
-Long id
-String name
-String address
-String email
-String phone
-LocalDateTime createdAt
manages
1
0..*
«Entity»
Group
-Long id
-String name
-String description
-LocalDate startDate
-LocalDate endDate
-boolean active
0..*
contains learners
0..*
TrainerReview
-Long id
-Long trainerId
-String comment
-Double adjustedGrammarScore
-Double adjustedLexiconScore
-Boolean validated
-LocalDateTime reviewedAt
trainerId
valide par
0..*
1
Feedback
-Long id
-Long recordingId
-Long userId
-String overallSummary
-Double grammarScore
-Double lexiconScore
-Double intonationScore
-String estimatedLevel
-String status
-LocalDateTime createdAt
userId
contient
1..*
Correction
-Long id
-String type
-String originalSegment
-String suggestedReformulation
-String explanation
-Boolean recurrentError
recommande
0..*
Exercise
-Long id
-Long userId
-String title
-String targetCriterion
-String targetLevel
-String instructions
-Boolean completed
userId
«Entity»
User
-Long id
-String firstName
-String lastName
-String email
-String keycloakId
-UserRole role
-String preferredLanguage
-boolean active
-LocalDateTime createdAt
userKeycloakId
«Entity»
TargetLevel
-Long id
-String language
-String certification
-CEFRLevel level
-LocalDate targetDate
-LocalDateTime createdAt
code
defines
0..*
1
«Entity»
ProgressSnapshot
-Long id
-Double pronunciationScore
-Double fluencyScore
-Double intonationScore
-Double vocabularyScore
-Double grammarScore
-Double coherenceScore
-Double overallScore
-CEFRLevel estimatedLevel
-LocalDateTime recordedAt
has history
0..*
«enumeration»
CEFRLevel
B1
B2
C1
C2
«Entity»
Alert
-Long id
-String title
-String message
-AlertType type
-boolean read
-LocalDateTime createdAt
receives
0..*
«enumeration»
AlertType
LOW_PROGRESS
LEVEL_GAP
INACTIVITY
TARGET_REACHED
«enumeration»
UserRole
APPRENANT
FORMATEUR
ETABLISSEMENT
Certification
-Long id
-String code
-String name
-String organization
-String language
delivre
0..*
1
evalue
CertificationLevel
-Long id
-CEFRLevel cecrlLevel
-Double passingScore
couvre
1..*
1
format
seuils
0..*
ExamFormatSpec
-Long id
-Int totalDurationMin
-Int preparationTimeMin
-Int nbParts
parties
1..*
ExamPart
-Long id
-Int partOrder
-String taskType
-Int durationMin
CriterionThreshold
-Long id
-String criterion
-Double minScore
Attestation
-Long id
-String reference
-String userKeycloakId
-Long simulationId
-CEFRLevel cecrlLevel
-Double globalScore
-LocalDateTime issuedAt
detail
1..*
AttestationCriterionScore
-Long id
-String criterion
-Double score
ReadinessReport
-Long id
-String userKeycloakId
-Double globalGap
-Double readinessPercent
-String status
0..*
simulationId
SimulationSession
-Long id
-String userKeycloakId
-Long topicId
-String topicTitle
-String topicPrompt
-CEFRLevel level
-String language
-ExamFormat format
-SimulationStatus status
-LocalDateTime prepEndsAt
-LocalDateTime startedAt
-LocalDateTime endedAt
-CEFRLevel estimatedLevel
-Double overallScore
«enumeration»
SimulationStatus
CREATED
PREPARING
IN_PROGRESS
PAUSED
FINISHED
ABANDONED
«enumeration»
ExamFormat
MONOLOGUE
INTERACTION
ROLE_PLAY
Topic
-Long id
-String title
-String prompt
-CEFRLevel level
-String language
-ExamFormat format
topicId
contains
0..*
SimulationTurn
-Long id
-int turnOrder
-TurnRole role
-String content
-Long audioId
-LocalDateTime createdAt
«enumeration»
TurnRole
EXAMINER
STUDENT
audioId
Recording
-Long id
-String fileName
-String transcript
-Int dureeSec
-LocalDateTime createdAt
recordingId
evaluated by
1
0..*
IntonationCurve
-List points
Analysis
-String id
-Long recordingId
-Long userId
-Date date
intonation
scores
mots
pauses
CriterionScore
-String critere
-Double score
WordResult
-String mot
-Double debut
-Double scoreProno nc
phonemes
Pause
-Double debut
-Int dureeMs
SimulationScore
-Long id
-Long criterionId
-Double score
-String comment
criterionId
Criterion
-Long id
-String name
-String descriptor
Grid
-Long id
-String niveau
-String langue
evalue
decrit
Descriptor
-Long id
-String texte

Diagramme de classe domaine



Diagramme de Classes du Domaine — Plateforme d'Entraînement et d'Évaluation de l'Expression Orale

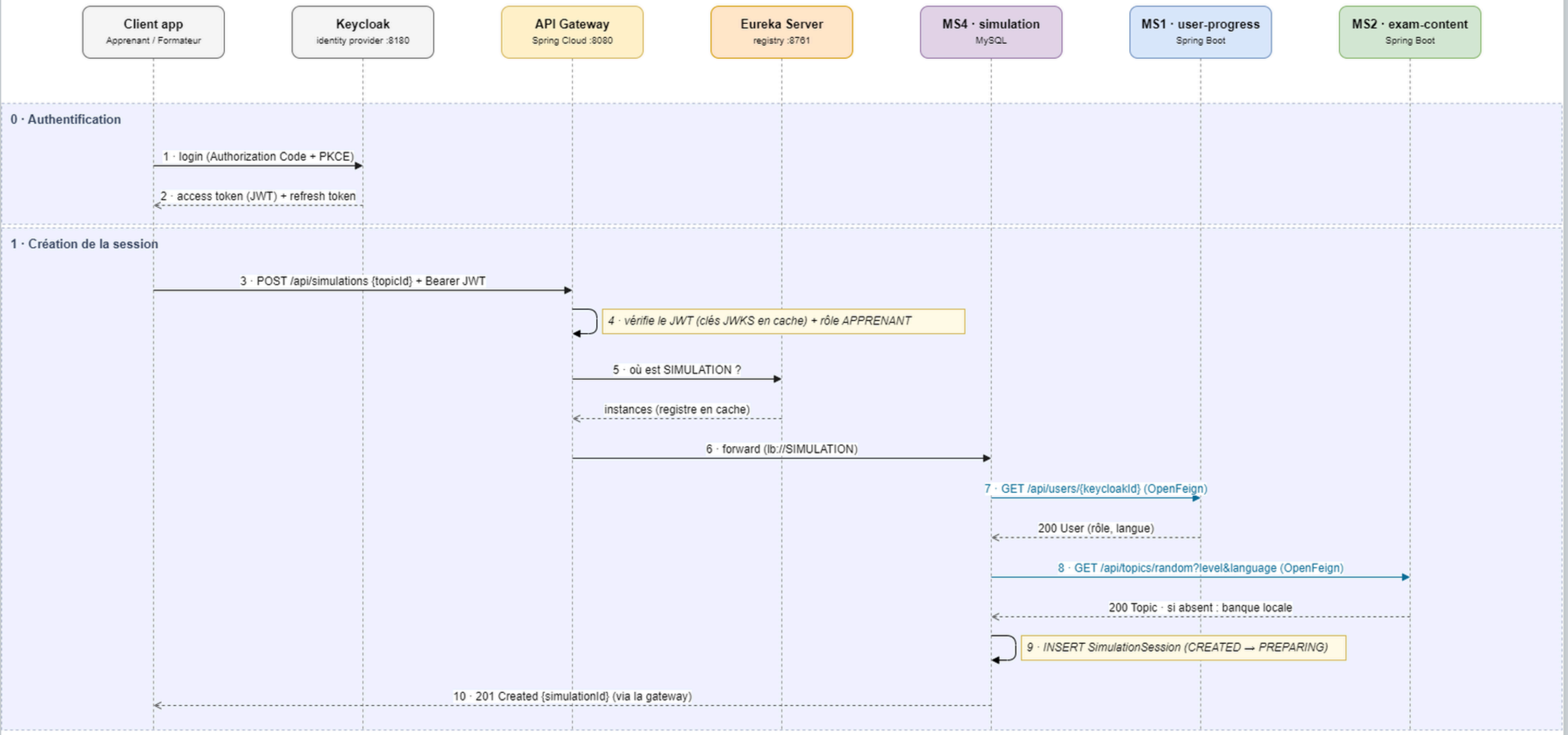


Synchronous communication (OpenFeign)



Séquence 1/4 · Créer la session d'examen (synchrone)

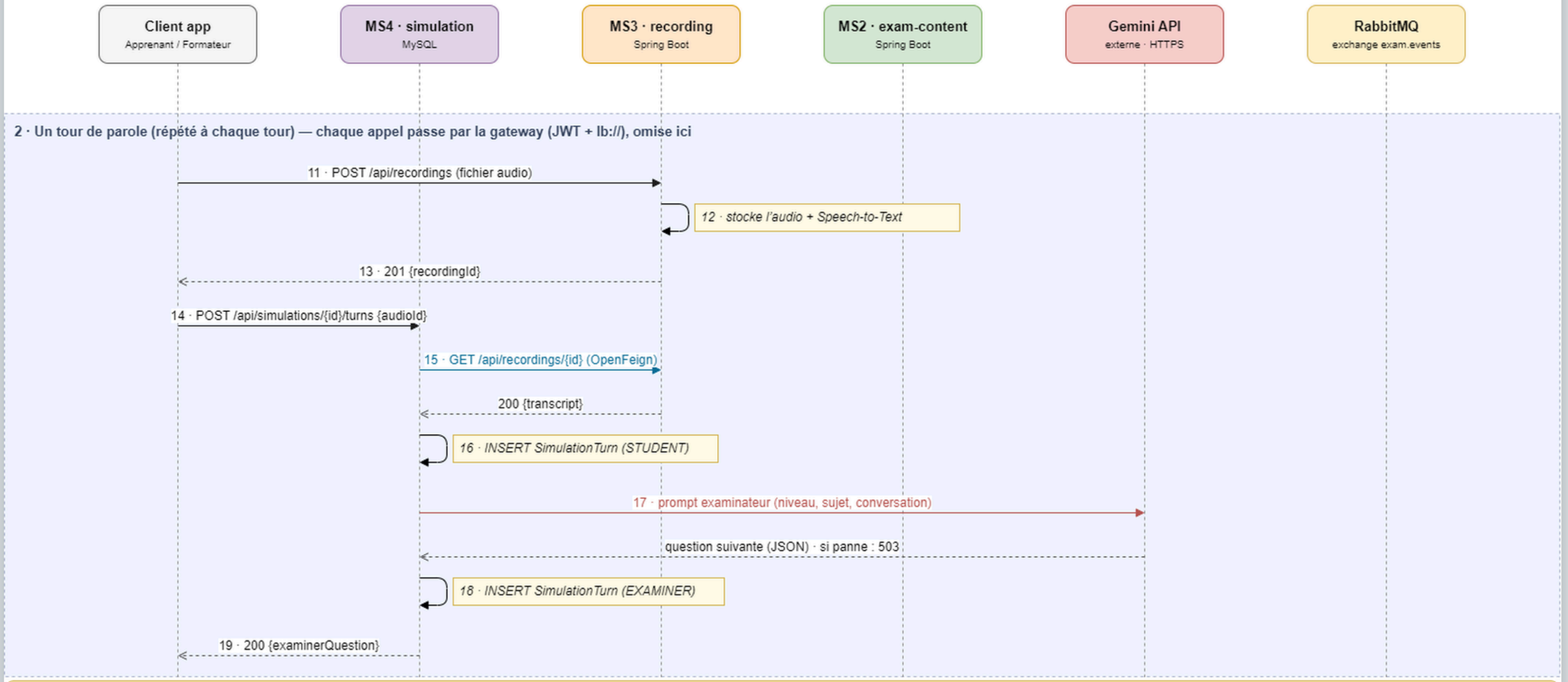
Noir = HTTP (client / gateway) · Bleu = appel synchrone OpenFeign · Rouge = API externe (Gemini) · Vert pointillé = événement asynchrone RabbitMQ · Gris pointillé = réponse



À retenir : étapes 1 à 10 synchrones, le client attend chaque réponse. Eureka est un annuaire, pas un proxy. Simulation ne garde que userKeycloakId et topicId : aucune clé étrangère vers un autre service.

Séquence 2/4 · Répondre et terminer l'épreuve (synchrone + Gemini)

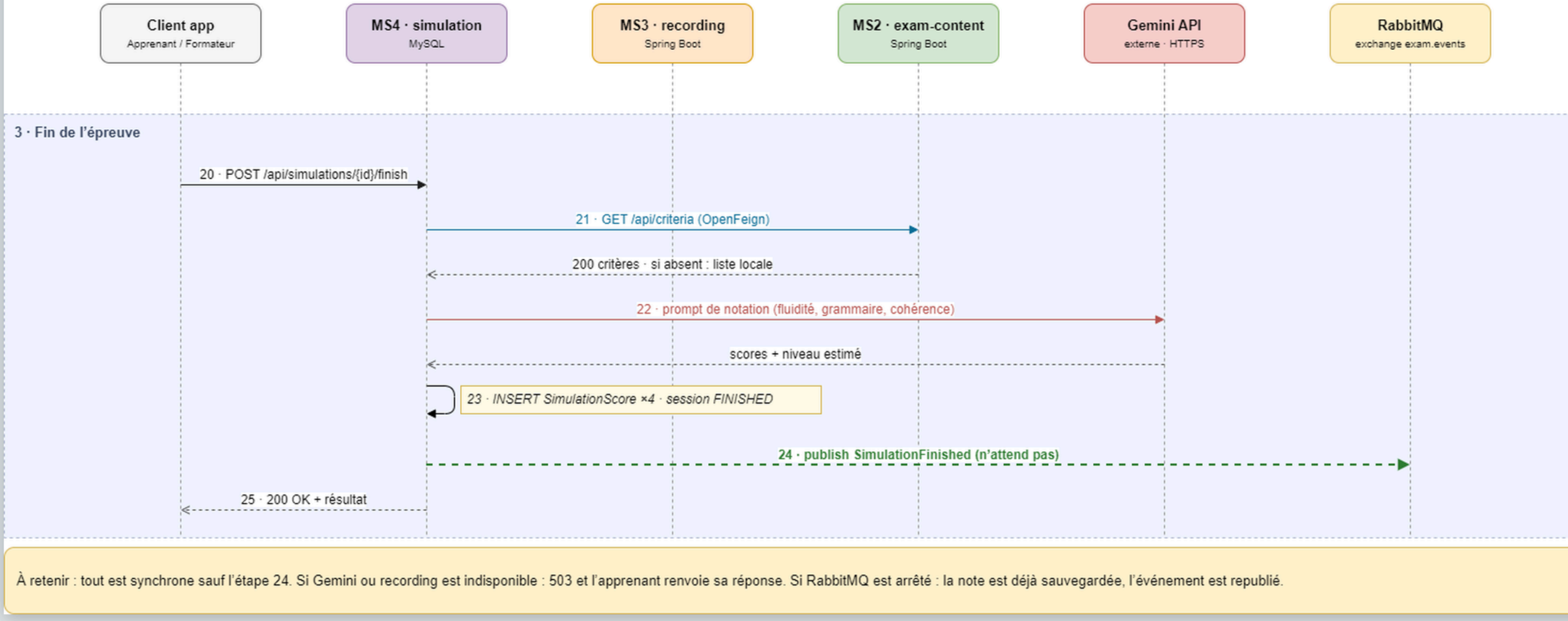
Noir = HTTP (client / gateway) · Bleu = appel synchrone OpenFeign · Rouge = API externe (Gemini) · Vert pointillé = événement asynchrone RabbitMQ · Gris pointillé = réponse



À retenir : tout est synchrone sauf l'étape 24. Si Gemini ou recording est indisponible : 503 et l'apprenant renvoie sa réponse. Si RabbitMQ est arrêté : la note est déjà sauvegardée, l'événement est republié.

Séquence 2/4 · Répondre et terminer l'épreuve (synchrone + Gemini)

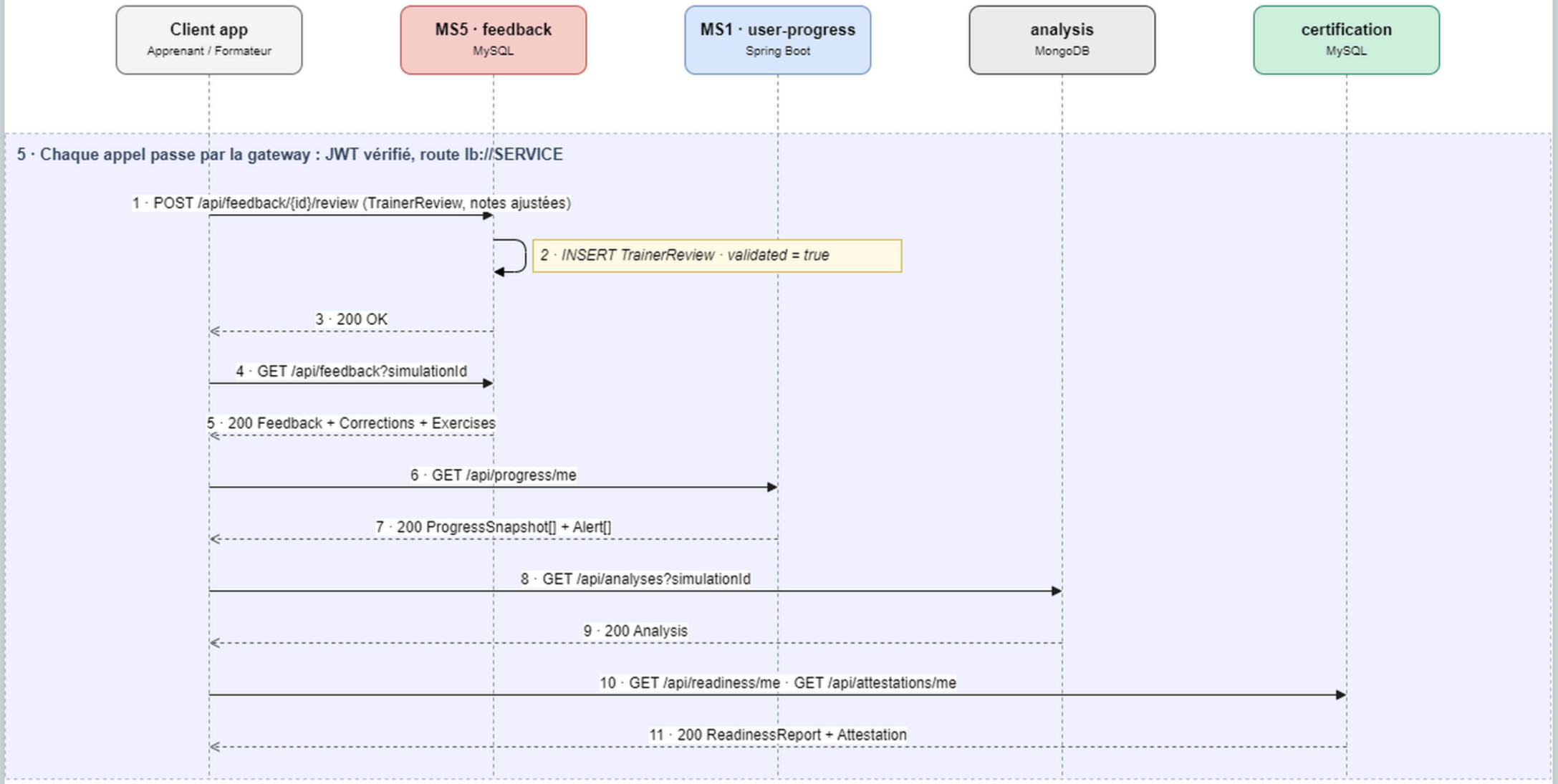
Noir = HTTP (client / gateway) · Bleu = appel synchrone OpenFeign · Rouge = API externe (Gemini) · Vert pointillé = événement asynchrone RabbitMQ · Gris pointillé = réponse



À retenir : tout est synchrone sauf l'étape 24. Si Gemini ou recording est indisponible : 503 et l'apprenant renvoie sa réponse. Si RabbitMQ est arrêté : la note est déjà sauvegardée, l'événement est republié.

Séquence 4/4 · Consultation et revue formateur (synchrone, via la gateway)

Noir = HTTP (client / gateway) · Bleu = appel synchrone OpenFeign · Rouge = API externe (Gemini) · Vert pointillé = événement asynchrone RabbitMQ · Gris pointillé = réponse



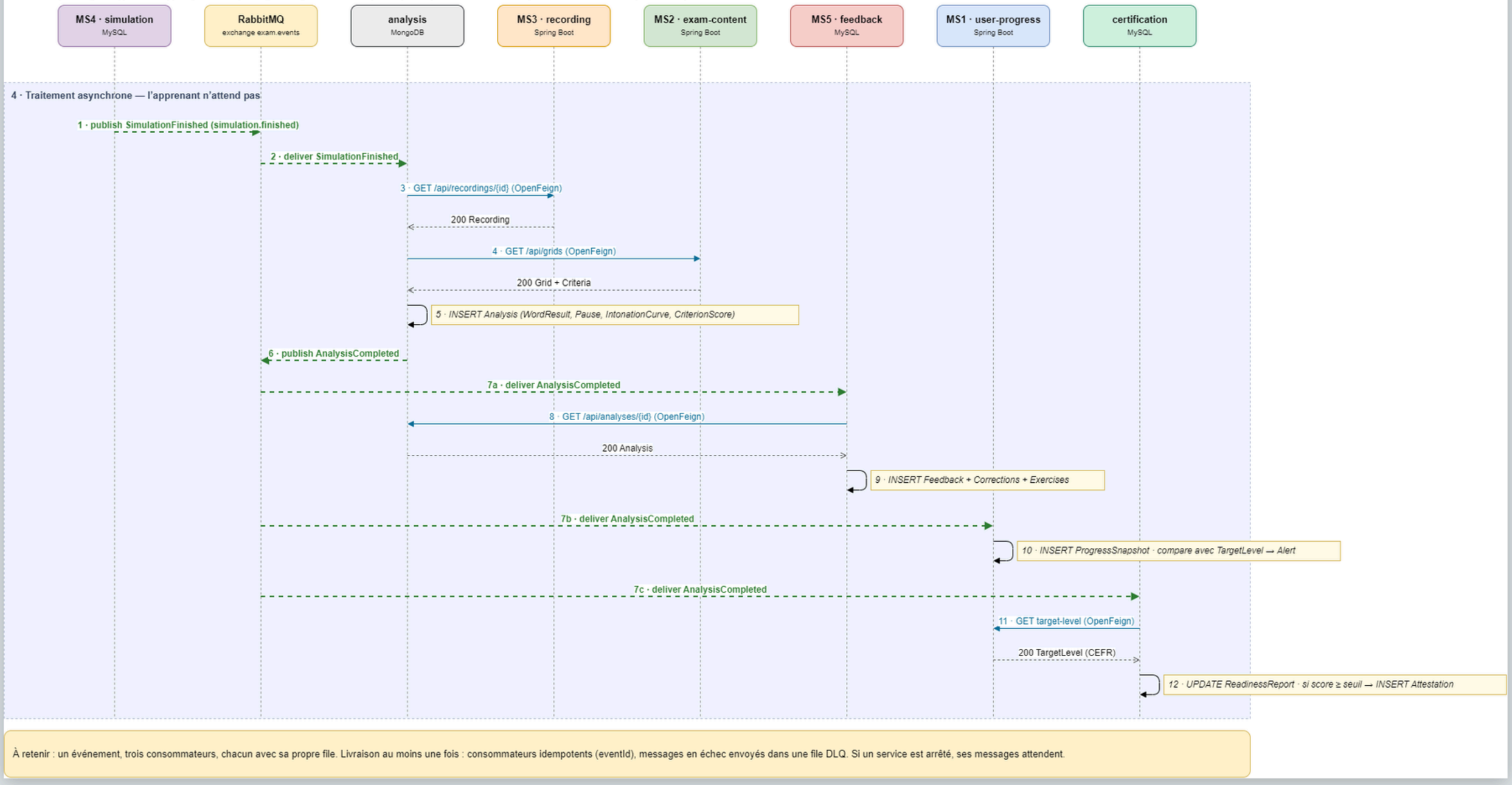
À retenir : 401/403 à la gateway (jeton absent ou mauvais rôle), 404 si la ressource n'existe pas, 503 si un service est arrêté ou absent d'Eureka. Seule la gateway est publique.

Asynchronous communication (RabbitMQ)

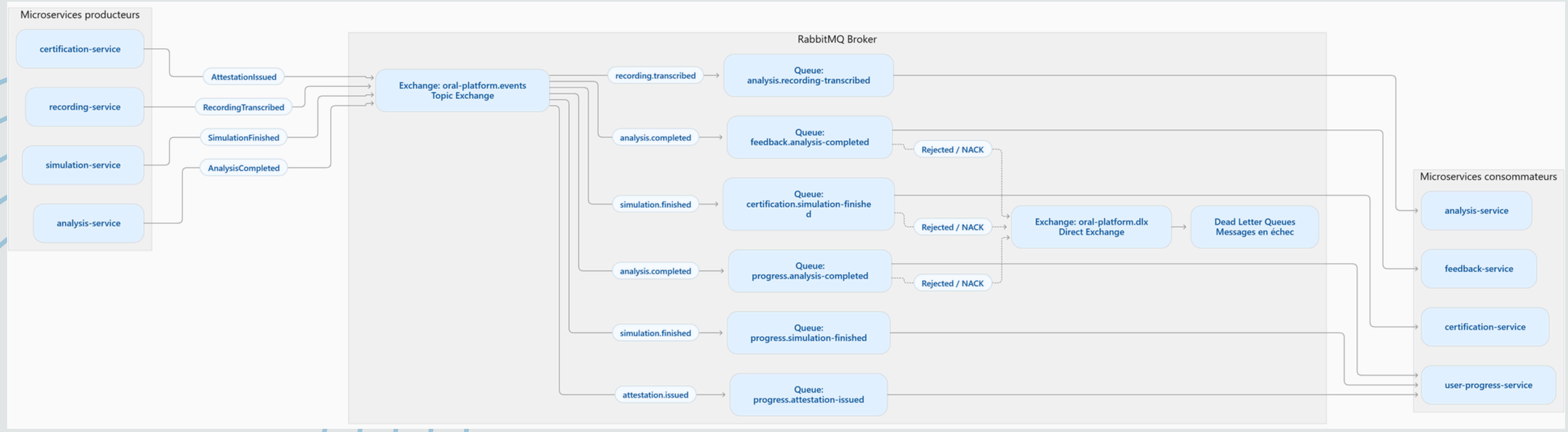


Séquence 3/4 · Analyse, feedback, progression, certification (asynchrone · RabbitMQ)

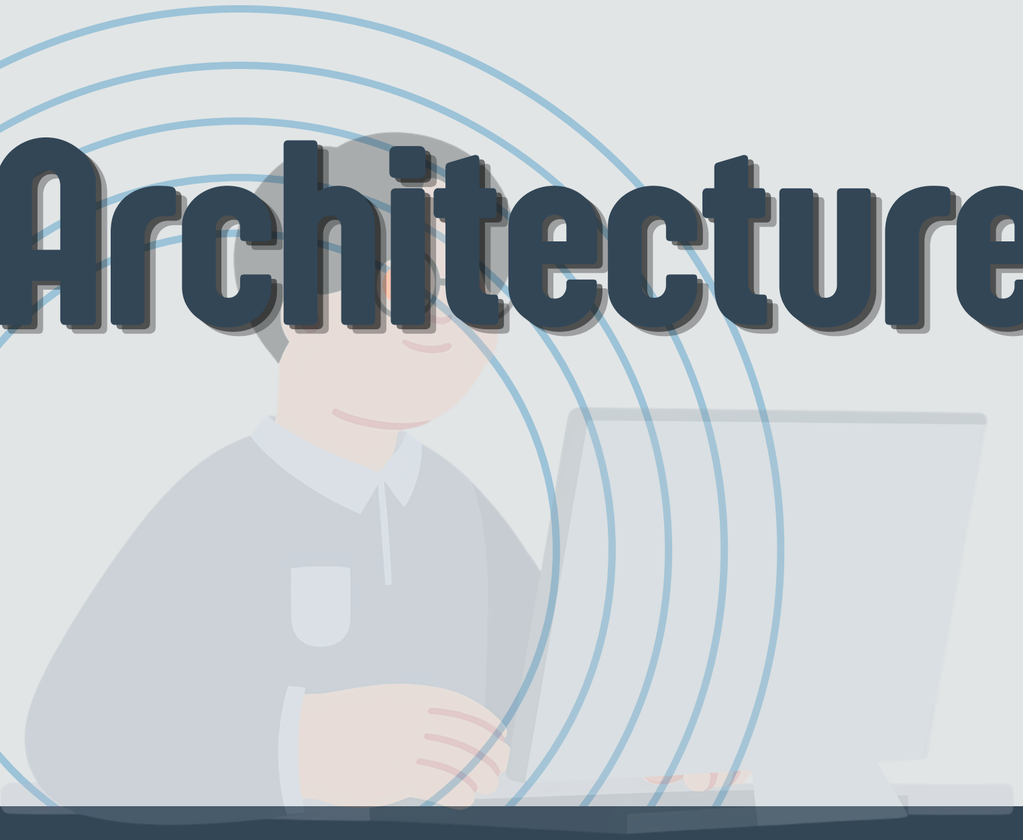
Noir = HTTP (client / gateway) · Bleu = appel synchrone OpenFeign · Rouge = API externe (Gemini) · Vert pointillé = événement asynchrone RabbitMQ · Gris pointillé = réponse

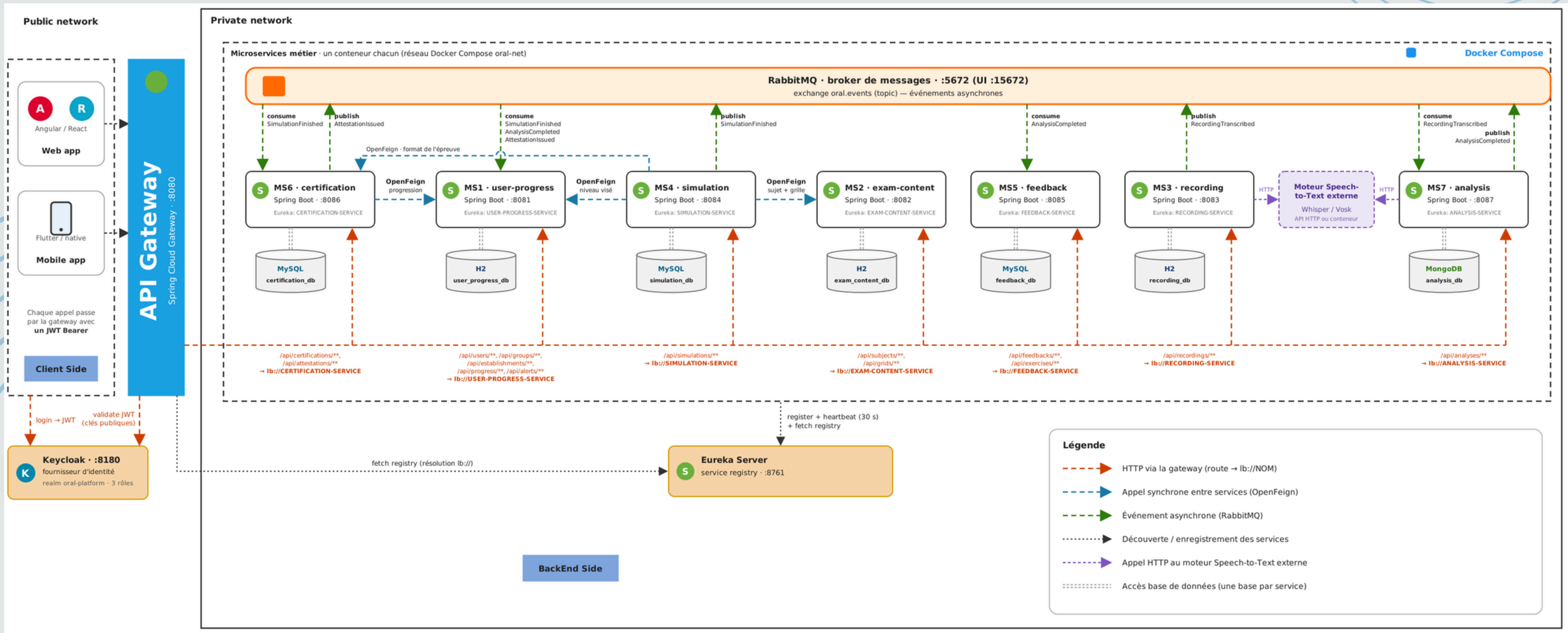


À retenir : un événement, trois consommateurs, chacun avec sa propre file. Livraison au moins une fois : consommateurs idempotents (eventId), messages en échec envoyés dans une file DLQ. Si un service est arrêté, ses messages attendent.

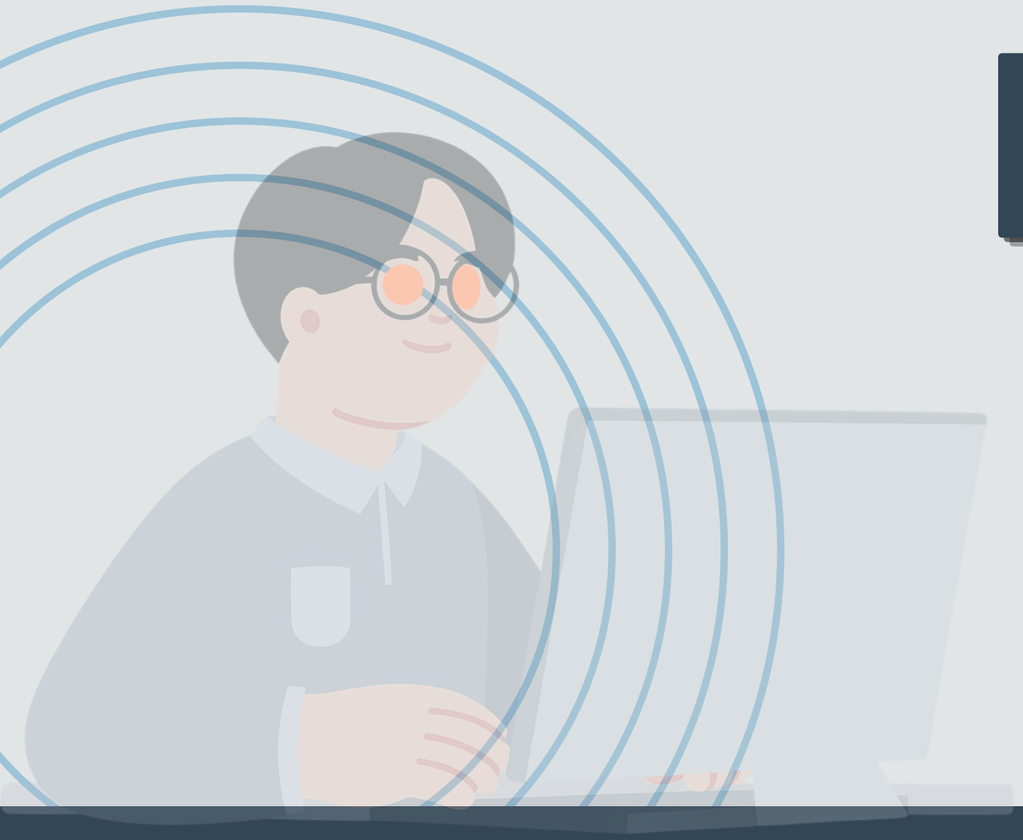

Microservices producteurs
certification-service
recording-service
simulation-service
analysis-service
AttestationIssued
RecordingTranscribed
SimulationFinished
AnalysisCompleted
RabbitMQ Broker
Exchange: oral-platform.events
Topic Exchange
recording.transcribed
Queue:
analysis.recording-transcribed
analysis.completed
Queue:
feedback.analysis-completed
Rejected / NACK
simulation.finished
Queue:
certification.simulation-finished
Rejected / NACK
analysis.completed
Queue:
progress.analysis-completed
Rejected / NACK
simulation.finished
Queue:
progress.simulation-finished
attestation.issued
Queue:
progress.attestation-issued
Exchange: oral-platform.dlx
Direct Exchange
Dead Letter Queues
Messages en échec
Microservices consommateurs
analysis-service
feedback-service
certification-service
user-progress-service

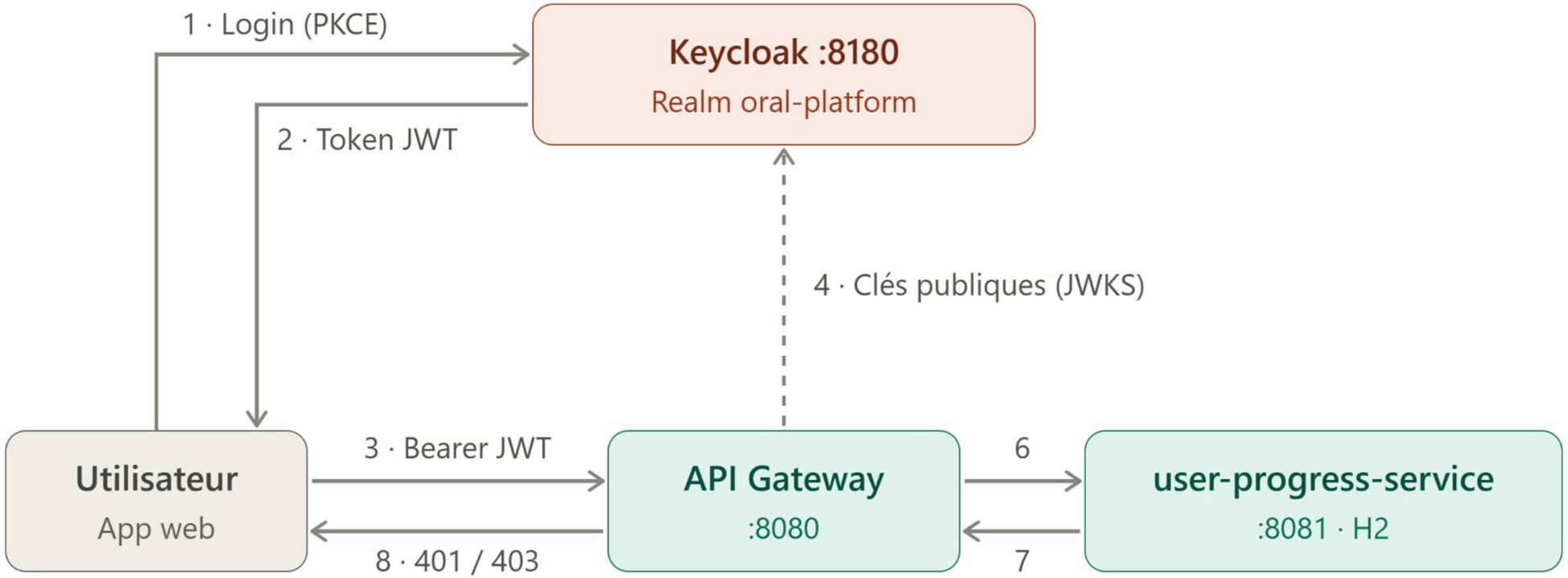
Architecture de la plateforme




Public network
Private network
Microservices métier · un conteneur chacun (réseau Docker Compose oral-net)
Docker Compose
A
R
Angular / React
Web app
Flutter / native
Mobile app
Chaque appel passe par la gateway avec un JWT Bearer
Client Side
API Gateway
Spring Cloud Gateway · :8080
RabbitMQ · broker de messages · :5672 (UI :15672)
exchange oral.events (topic) — événements asynchrones
consume SimulationFinished
publish AttestationIssued
consume SimulationFinished AnalysisCompleted AttestationIssued
publish SimulationFinished
consume AnalysisCompleted
publish RecordingTranscribed
consume RecordingTranscribed
publish AnalysisCompleted
OpenFeign · format de l'épreuve
MS6 · certification
Spring Boot · :8086
Eureka: CERTIFICATION-SERVICE
OpenFeign progression
MS1 · user-progress
Spring Boot · :8081
Eureka: USER-PROGRESS-SERVICE
OpenFeign niveau visé
MS4 · simulation
Spring Boot · :8084
Eureka: SIMULATION-SERVICE
OpenFeign sujet + grille
MS2 · exam-content
Spring Boot · :8082
Eureka: EXAM-CONTENT-SERVICE
MS5 · feedback
Spring Boot · :8085
Eureka: FEEDBACK-SERVICE
MS3 · recording
Spring Boot · :8083
Eureka: RECORDING-SERVICE
HTTP
Moteur Speech-to-Text externe
Whisper / Vosk
API HTTP ou conteneur
HTTP
MS7 · analysis
Spring Boot · :8087
Eureka: ANALYSIS-SERVICE
MySQL certification_db
H2 user_progress_db
MySQL simulation_db
H2 exam_content_db
MySQL feedback_db
H2 recording_db
MongoDB analysis_db
/api/certifications/**, /api/attestations/** → lb://CERTIFICATION-SERVICE
/api/users/**, /api/groups/**, /api/establishments/**, /api/progress/**, /api/alerts/** → lb://USER-PROGRESS-SERVICE
/api/simulations/** → lb://SIMULATION-SERVICE
/api/subjects/**, /api/grids/** → lb://EXAM-CONTENT-SERVICE
/api/feedbacks/**, /api/exercises/** → lb://FEEDBACK-SERVICE
/api/recordings/** → lb://RECORDING-SERVICE
/api/analyses/** → lb://ANALYSIS-SERVICE
login → JWT
validate JWT (clés publiques)
Keycloak · :8180
fournisseur d'identité
realm oral-platform · 3 rôles
register + heartbeat (30 s) + fetch registry
fetch registry (résolution lb://)
Eureka Server
service registry · :8761
BackEnd Side
Légende
HTTP via la gateway (route → lb://NOM)
Appel synchrone entre services (OpenFeign)
Événement asynchrone (RabbitMQ)
Découverte / enregistrement des services
Appel HTTP au moteur Speech-to-Text externe
Accès base de données (une base par service)

keykloak





5 · La gateway vérifie signature, expiration, issuer et rôle

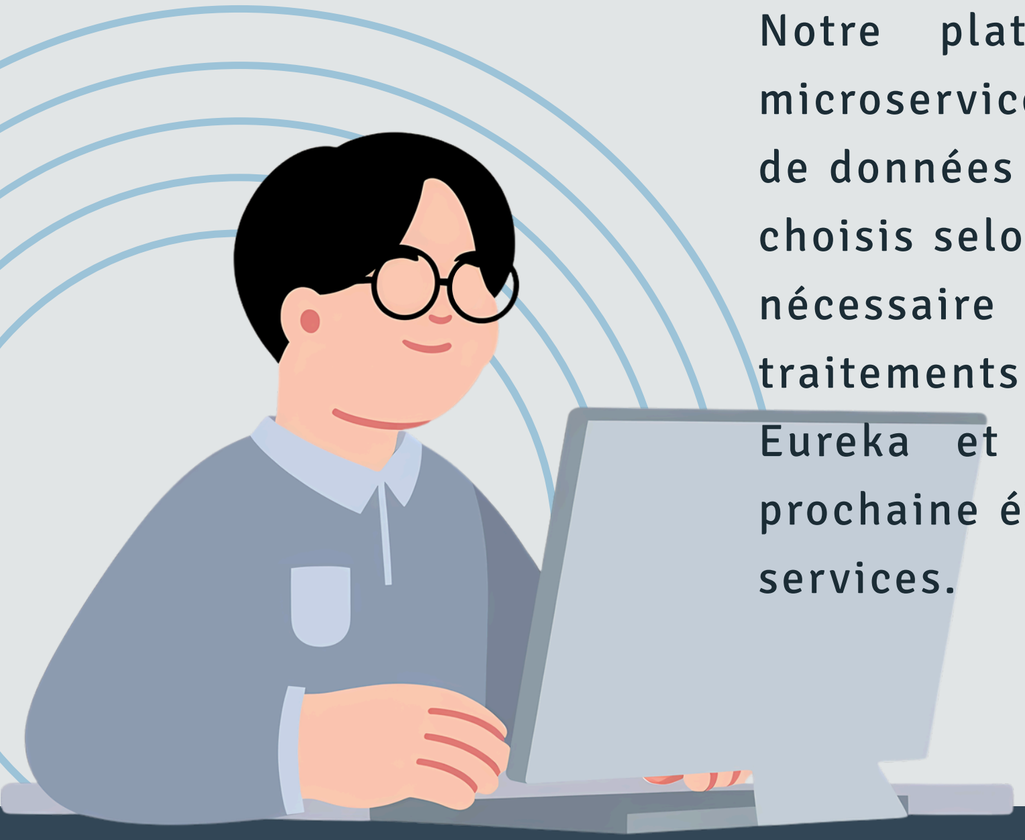
Accès selon le rôle du JWT

APPRENANT
Sa progression

FORMATEUR
Dashboard des groupes

ETABLISSEMENT
Indicateurs agrégés

Conclusion



Notre plateforme d’entraînement à l’oral est découpée en microservices selon les capacités métier, chacun avec sa propre base de données et ne partageant que des identifiants. Les échanges sont choisis selon le besoin : OpenFeign (synchrone) quand la réponse est nécessaire immédiatement, RabbitMQ (asynchrone) pour les traitements longs comme l’analyse, avec l’API Gateway, Keycloak, Eureka et le Config Server pour compléter l’architecture. La prochaine étape est de développer la logique métier et d’intégrer les services.

MERCI POUR VOTRE ATTENTION

